

Face Recognition & Metric Learning

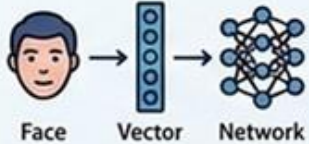
One shot learning
Siamese Networks
Triplet Loss
Embedding Mantığı

Defne BENLİYAY
10. Hafta

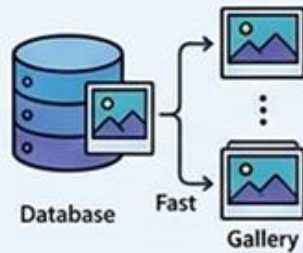
SUNUM VE AKIŞ PLANI

Giriş

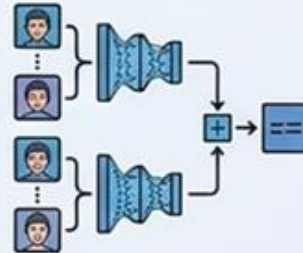
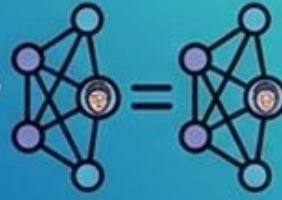
(Face Recognition, Metrik Learning, Embedding)



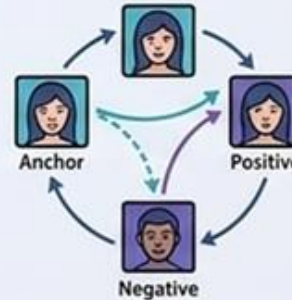
One-Shot Learning



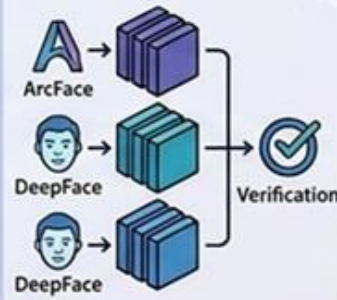
Siamese Networks



Triplet Loss

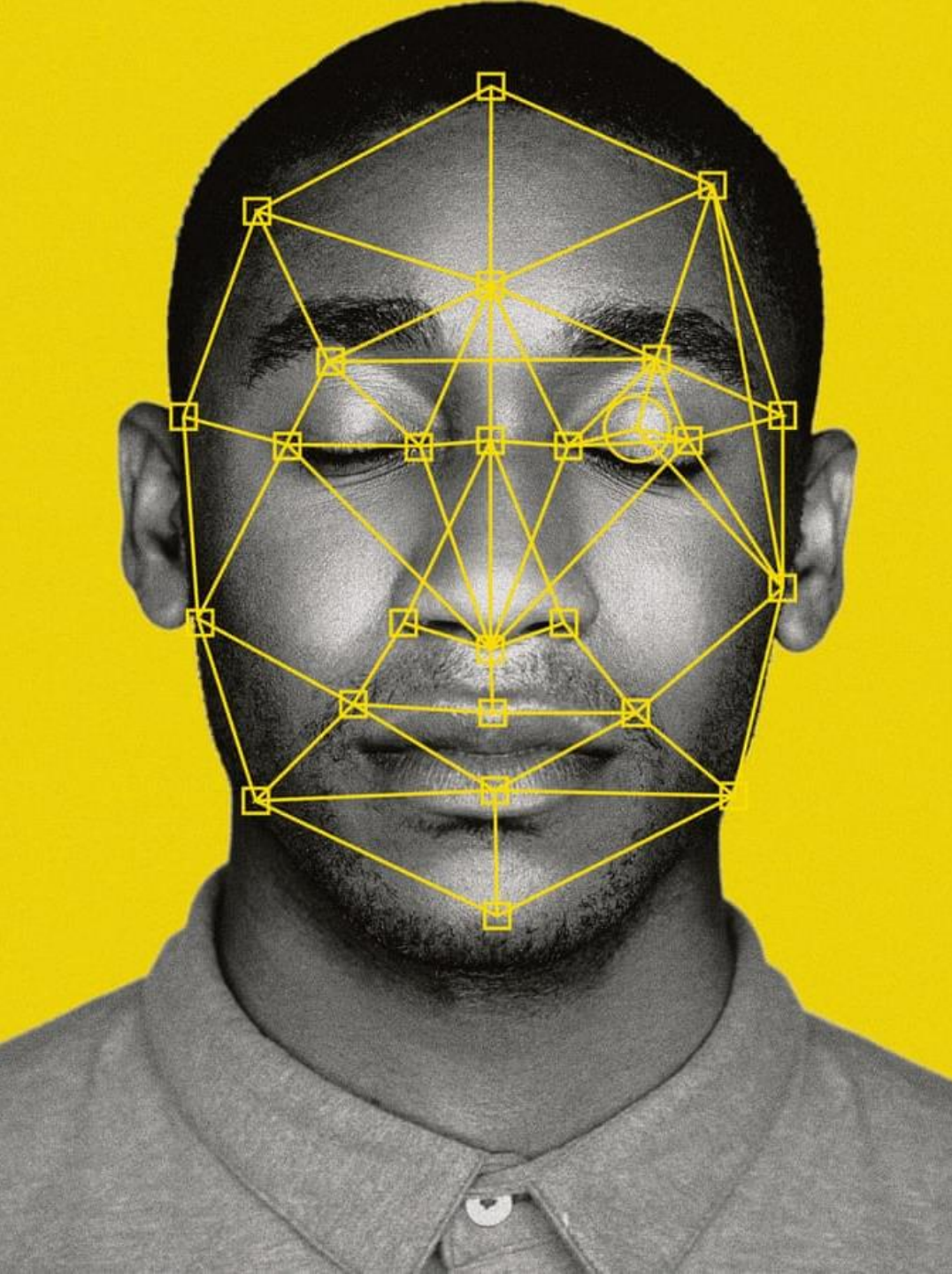


Yüz Tanıma Algoritmaları



Zorluklar ve Etik





Face Recognition (Yüz Tanıma)

Tanıma işlemi, tespitle başlar ama kimlik tespitiyle biter.

Face Detection (Yüz Tespiti): Görüntüde bir yüz var mı? (Konum belirleme).

Face Recognition (Yüz Tanıma): Bu yüz kime ait? (Kimlik eşleştirme).

Süreç: Önce yüz bulunur, sonra özellikleri çıkarılır, en son veri tabanıyla kıyaslanır.

Embedding Mantığı (Dijital Parmak İzi)

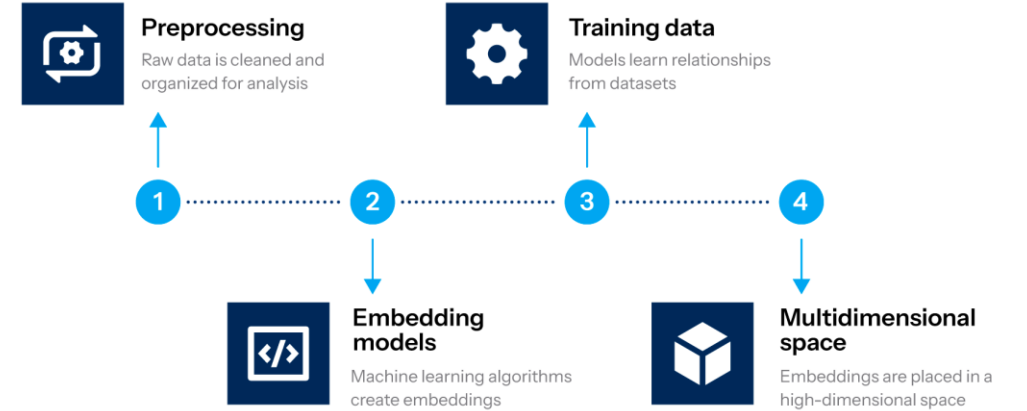
Görüntüden matematiksel bir özet çıkarma.

Vektör Dönüşümü: Resmin (örneğin 128 adet) sayıya indirgenmesi.

Anlamsal Temsil: Sayılar göz rengi, burun yapısı, çene hattı gibi özellikleri temsil eder.

Karşılaştırma: İki resmin ne kadar benzediği, bu iki sayı dizisi arasındaki matematiksel farktır.

How does vector embedding work?



Neden Geleneksel Sınıflandırma (Softmax) Kullanmıyoruz?

Klasik modeller milyonlarca kişiyi tanıyamaz.

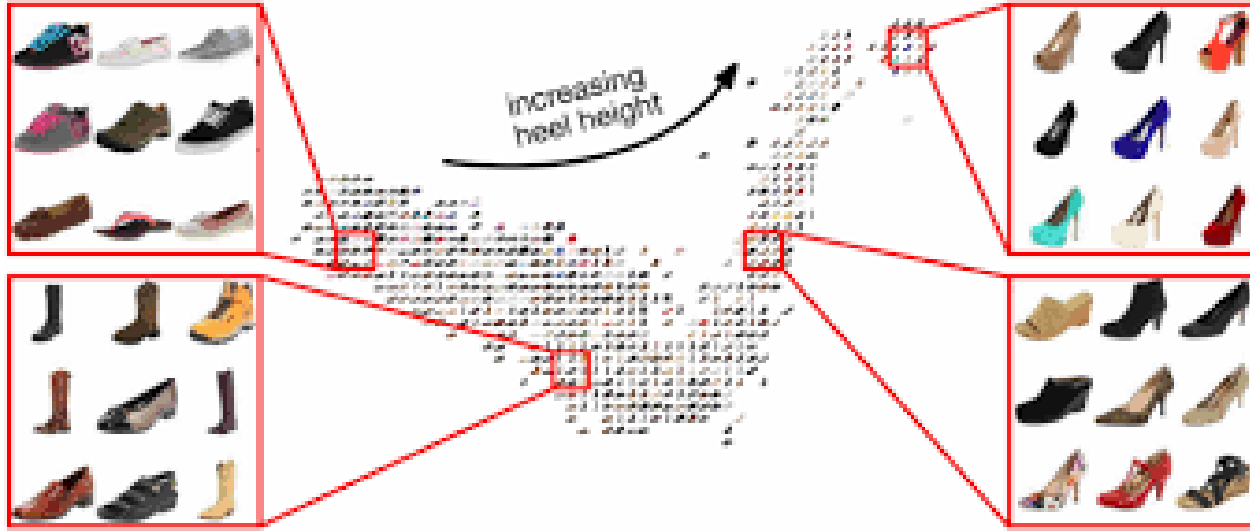
Ölçekleme Sorunu: 1 milyon kişi için son katmanda 1 milyon nöron gerekir.

Yeniden Eğitim: Sisteme yeni bir kişi katıldığında tüm modelin en baştan eğitilmesi zorunluluğu.

Veri Kıtlığı: Sınıflandırma için binlerce resim gerekirken, yüz tanımda elimizde genellikle tek bir resim vardır.



Metric Learning (Metrik Öğrenme) Nedir?



Kimlik sorma, benzerliği ölç.

Mantık: Resimleri yüksek boyutlu bir vektör uzayına yerleştirme.

Hedef: Aynı kişileri **yakın**, farklı kişileri **uzak** konumlara itmek.

Çıktı: Bir "sınıf" ismi değil, bir "mesafe" skorudur.

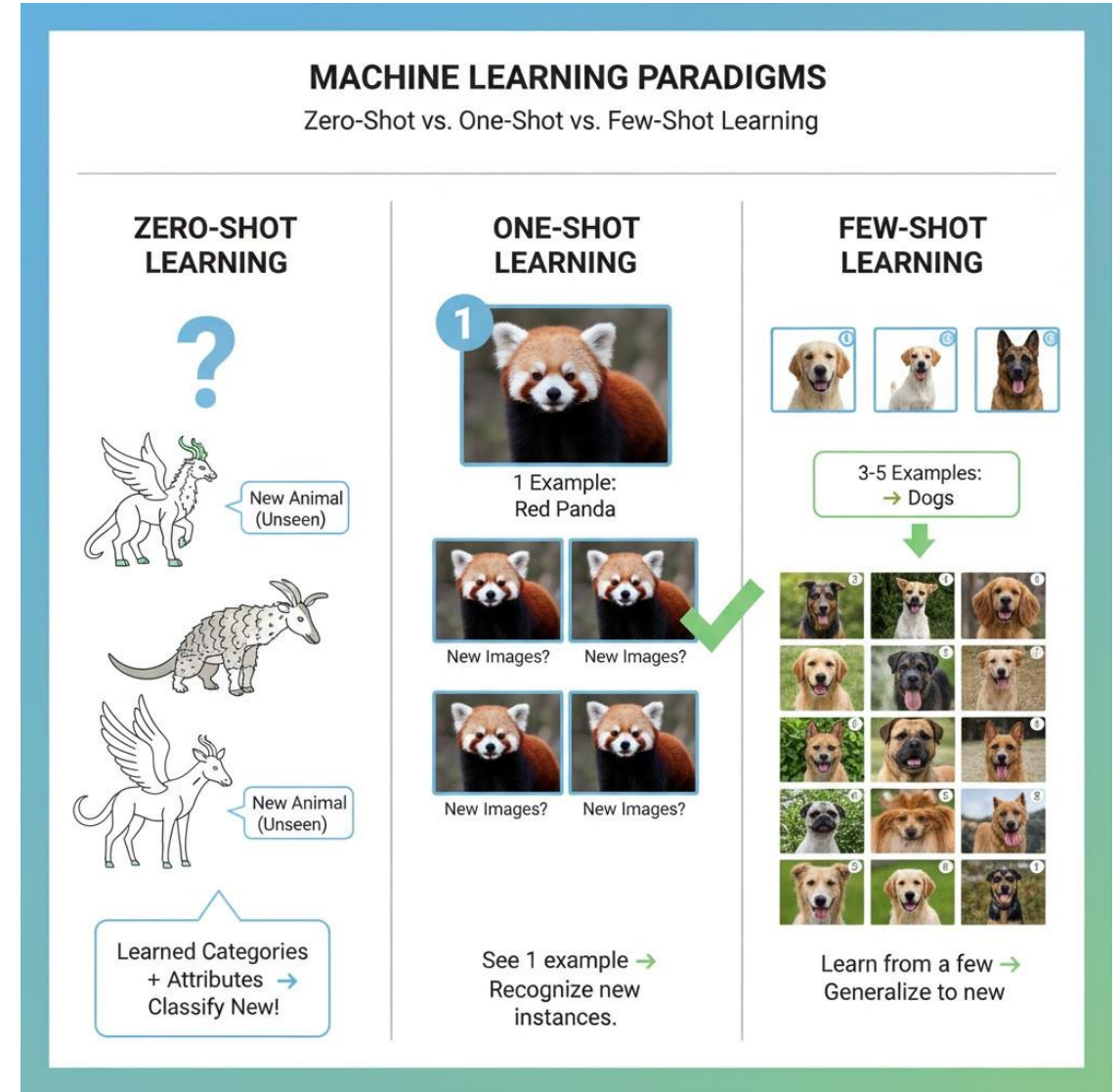
One-Shot Learning

Tek bir örnekle tanıma yeteneği.

Kavram: Bir nesneyi/yüzü sadece bir kez görerek tanıyabilme.

Önem: Personel takibinde her çalışanın binlerce fotoğrafına sahip olamayız.

Yöntem: Modeli "kategorileri" değil, "benzerlikleri" öğrenecek şekilde eğitmek.

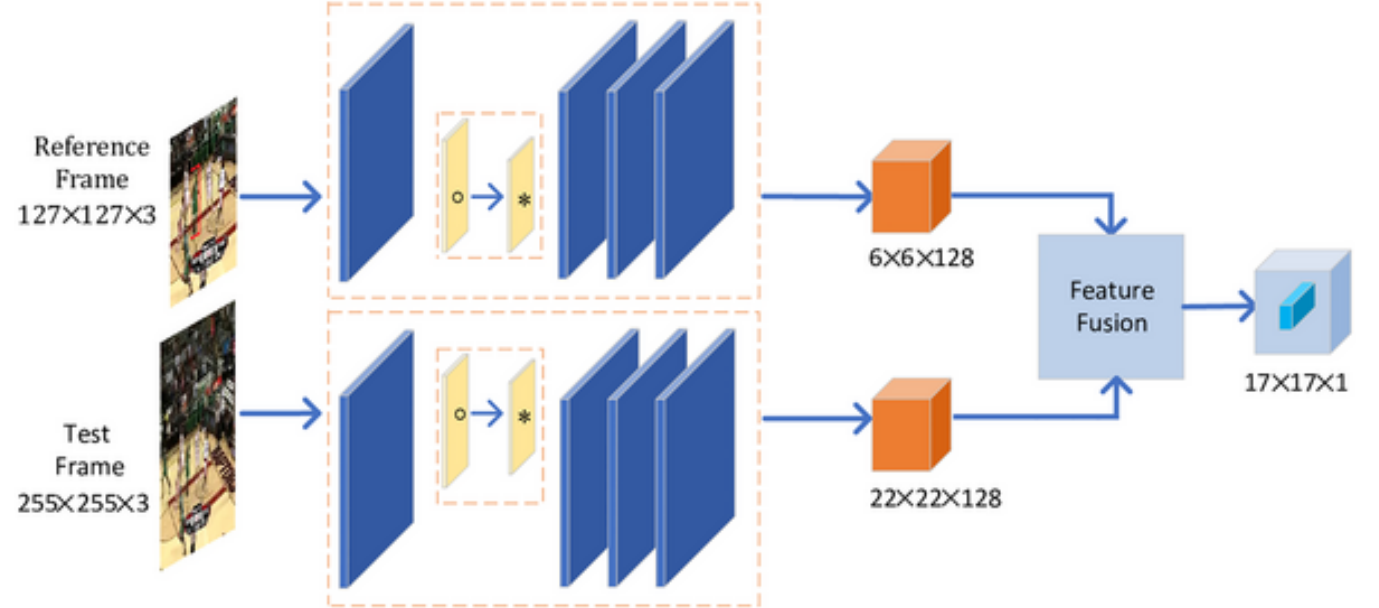


Siamese Networks

İsim Kökeni: Yapışık ikizlerden (Siyam ikizleri) gelir.

Yapı: Birbirinin tıpatıp aynısı olan iki ayrı sinir ağı kanalı.

Görev: İki farklı görüntüyü aynı anda işleyip, bunların özelliklerini (embedding) karşılaştırmak.

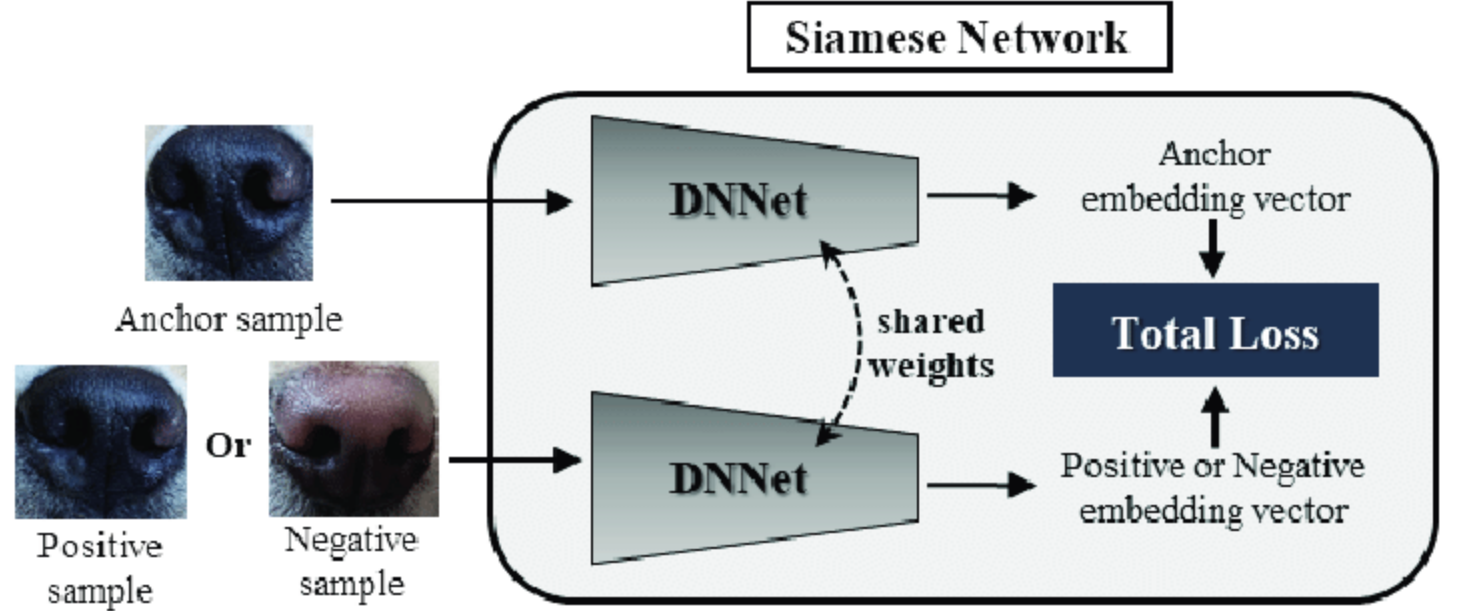


Çalışma Prensibi

Shared Weights (Paylaşımlı Ağlıklar):

Her iki kanal da aynı parametreleri kullanır. Bu, her iki resmin de aynı "bakış açısıyla" analiz edilmesini sağlar

Çıktı: Her iki resim için birer vektör üretilir ve aralarındaki mesafe (örneğin Öklid mesafesi) hesaplanır.



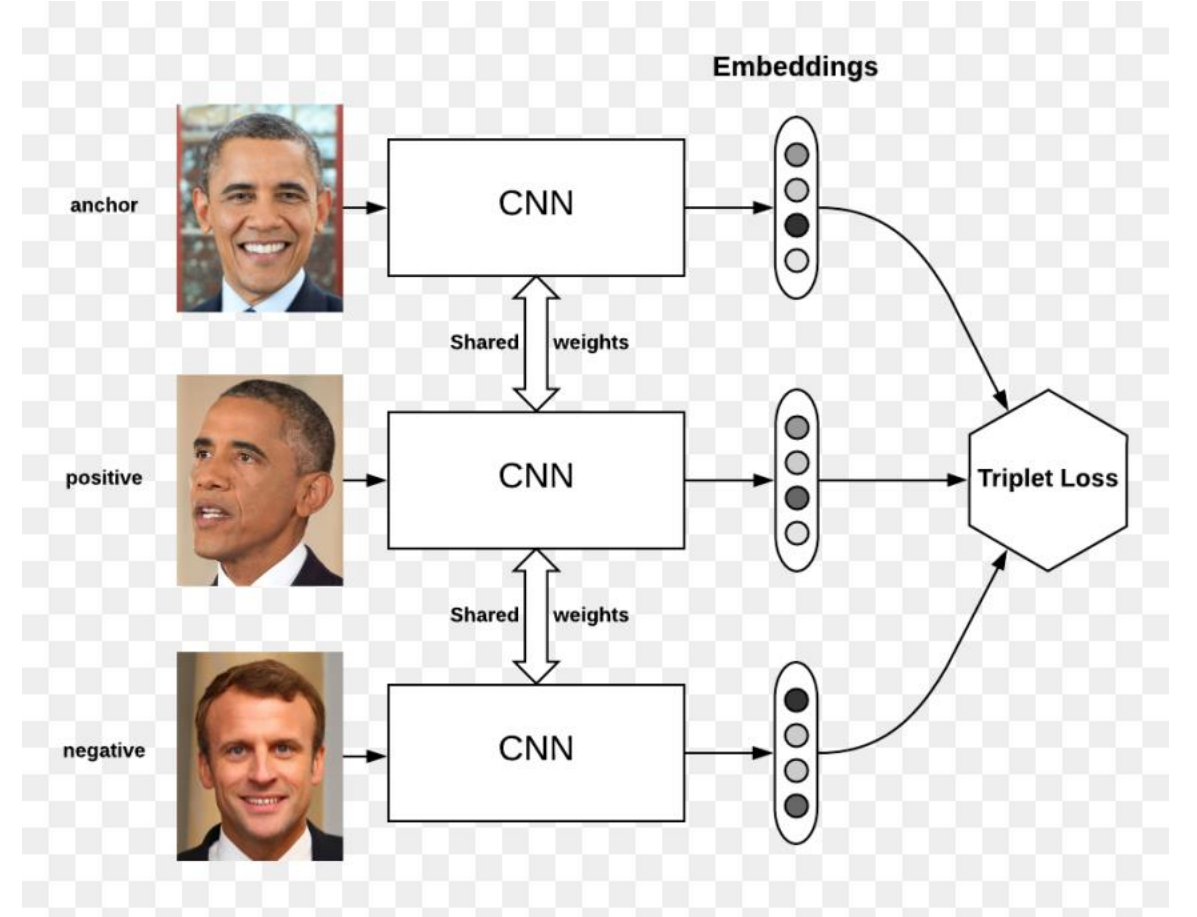
Triplet Loss Mantığı

Neden Gereklidir?

Sadece ikili karşılaştırma (pozitif/negatif) bazen modelin verimli öğrenmesi için yeterli olmaz.

Yöntem: Sistemi aynı anda üçlü gruplar üzerinden eğitiriz.

Amaç: Benzer olanı itmekten ziyade, "benzer olanın, benzemeyenden çok daha yakın" olmasını sağlamak.



Matematiksel Hedef ve Marjin

Anchor ile Positive arasındaki mesafe, Anchor ile Negative arasındaki mesafeden en az **alpha (marjin)** kadar küçük olmalıdır.

Marjin: Modelin "bu kadar fark bana yetmez, daha da uzaklaştır" demesini sağlayan güvenlik payıdır.

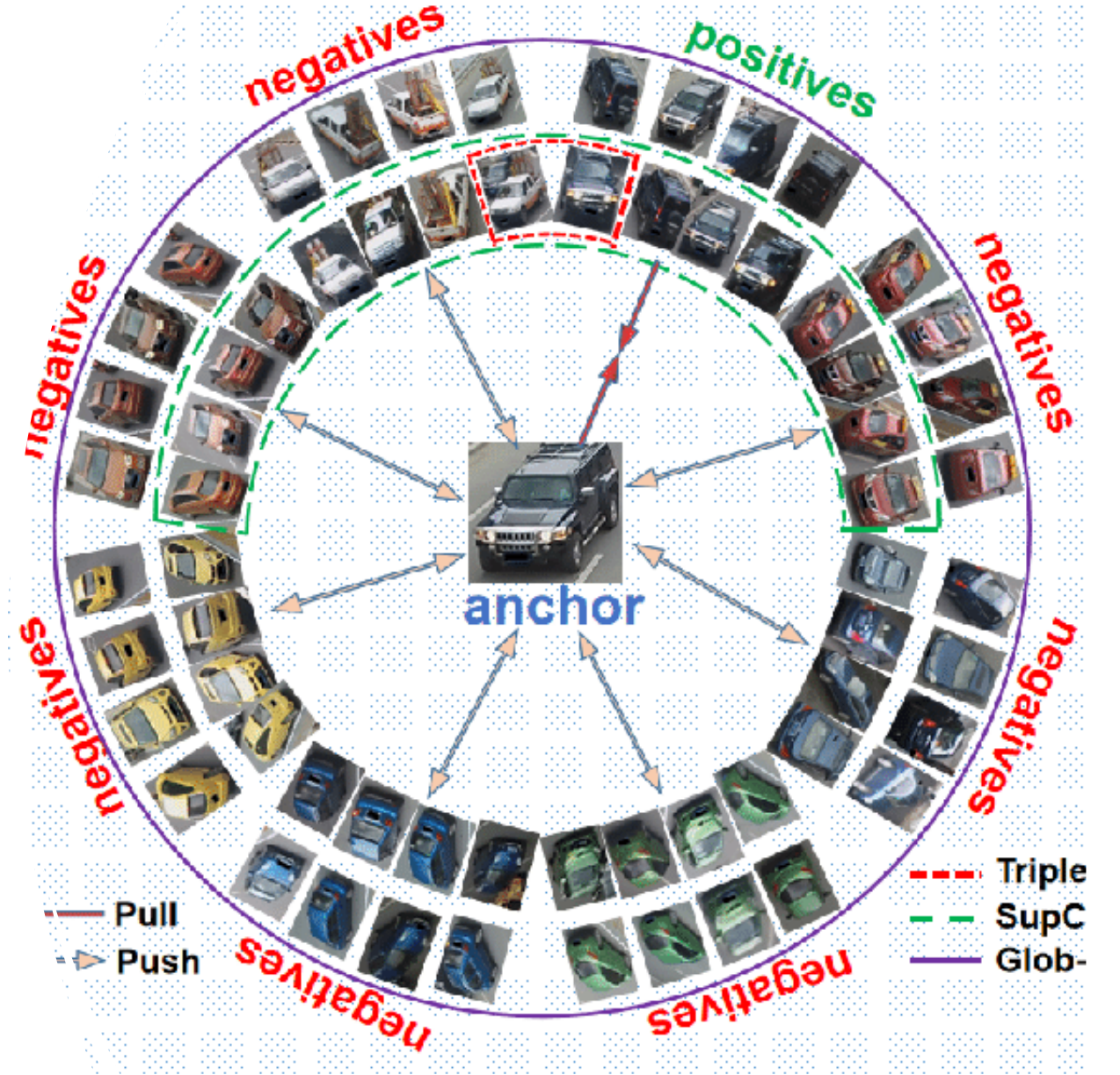
$$Loss = \sum_{i=1}^N \left[\|f_i^a - f_i^p\|_2^2 - \|f_i^a - f_i^n\|_2^2 + \alpha \right]_+$$

Örnek Seçim Stratejileri (Mining)

Easy Triplets: Modelin zaten doğru bildiği kolay örnekler (Eğitime katkısı az).

Hard Triplets: Modelin "benzer" sandığı ama aslında farklı olan zor örnekler.

Semi-Hard: Margin sınırında kalan, modeli en çok geliştiren "eğitici" örnekler.



Yüz Tanıma Algoritmaları

The image shows a chalkboard with handwritten mathematical derivations. At the top left, the equation $y = g(x)$ is written and crossed out. Below it, the words "Secant Lines" are written. To the right, the derivative of $f(x)$ is defined as a limit: $f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$. Below this, the specific function $f(x) = x^2$ is used to calculate the derivative: $f'(x) = \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h}$. This is then expanded to $\lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h}$, which simplifies to $\lim_{h \rightarrow 0} \frac{2xh + h^2}{h}$. Finally, it simplifies to $\lim_{h \rightarrow 0} h(2x + h)$. On the left side of the chalkboard, there are some additional notes and a small diagram showing a point $x+h$ on a horizontal axis.

$y = g(x)$

Secant Lines

$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$

$f(x) = \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h}$

$= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h}$

$= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h}$

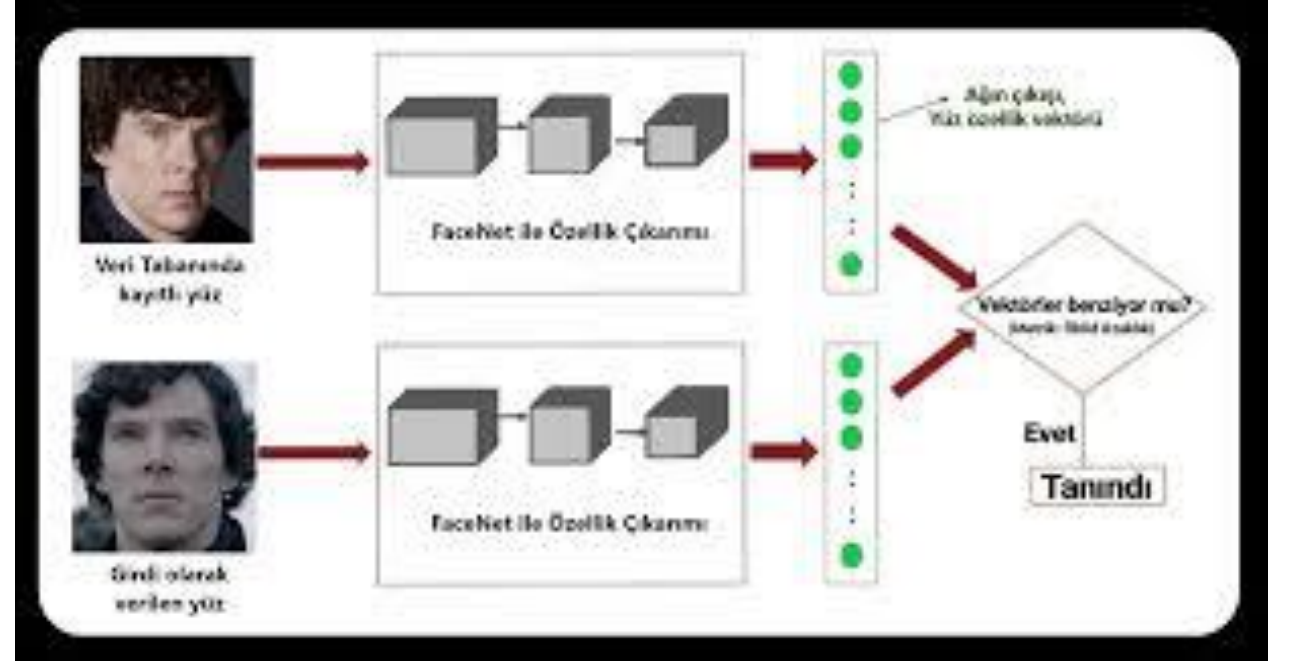
$= \lim_{h \rightarrow 0} h(2x + h)$

$x+h$

FaceNet (Google)

2015 yılında tanıtıldı. Triplet Loss kullanarak 128 boyutlu embedding üretimiyle standartları belirledi.

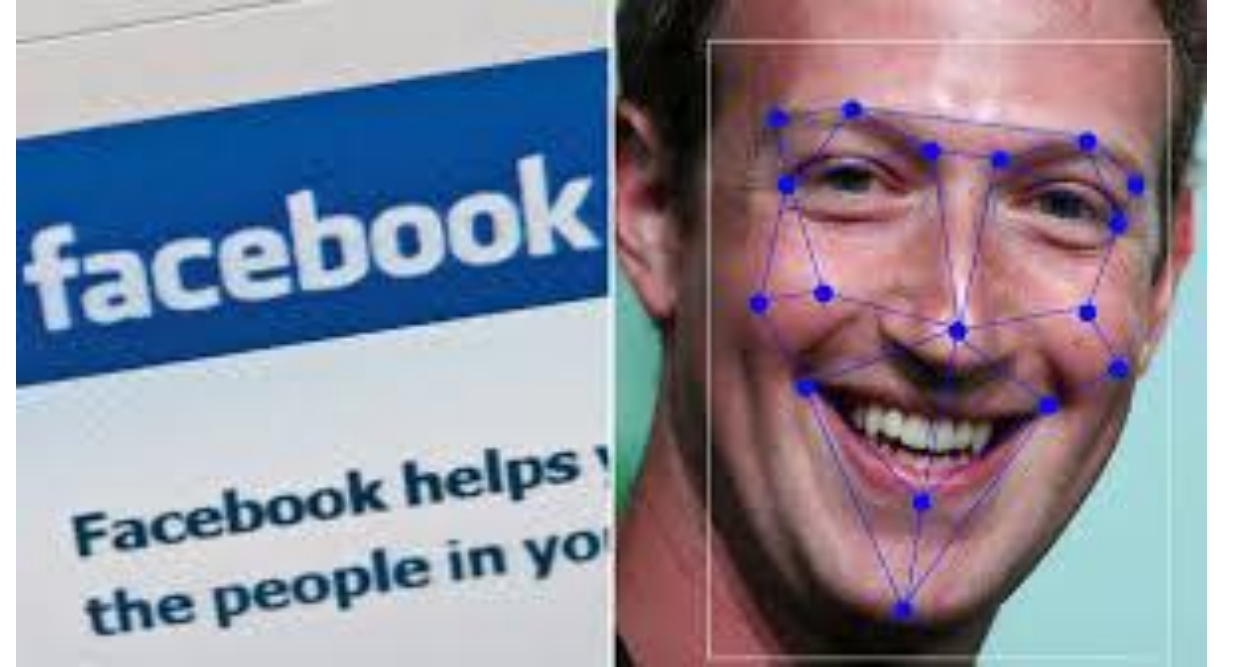
LFW veri setinde %99.63 gibi rekor bir başarıya ulaştı.



DeepFace (Facebook)

3D yüz hizalama kullanarak tanıma yapar. DeepFace, standart bir veri seti olan Labeled Faces in the Wild (LFW) üzerinde %97,35 doğruluk oranına ulaşarak, insanların ulaştığı %97,53'lük seviyeye yaklaşmıştır.

Algoritma, Facebook kullanıcıları tarafından yüklenen 4.000 farklı kişiye ait 4 milyon görüntü üzerinde eğitilmiştir.



ArcFace

"Additive Angular Margin Loss" kullanarak embedding uzayındaki sınıfları çok daha keskin çizgilerle ayırır.

Standart yöntemlerin aksine, yüz özelliklerini bir küre yüzeyindeki açılar olarak hesaplar. Aynı kişiye ait yüzleri birbirine yaklaştırırken, farklı kişilerin yüzlerini birbirinden uzaklaştırarak net bir ayırım sağlar.

they are different persons
Distance is 5.47 whereas as expected max threshold is 4.16



they are different persons
Distance is 5.7 whereas as expected max threshold is 4.16



they are different persons
Distance is 5.09 whereas as expected max threshold is 4.16



they are different persons
Distance is 5.42 whereas as expected max threshold is 4.16



they are different persons
Distance is 5.77 whereas as expected max threshold is 4.16



they are different persons
Distance is 5.05 whereas as expected max threshold is 4.16



they are different persons
Distance is 5.32 whereas as expected max threshold is 4.16



they are different persons
Distance is 5.76 whereas as expected max threshold is 4.16



Özellik	FaceNet	DeepFace	ArcFace
Geliştirici	Google	Facebook	InsightFace / DeepInsight
Temel Yöntem	Triplet Loss (Mesafe)	3D Alignment + CNN	Angular Margin (Açı)
Embedding Boyutu	Genellikle 128	4096	Genellikle 512
Güçlü Yanı	Verimli ve hızlı karşılaştırma	Açı farklarını 3D ile çözme	Çok yüksek ayırt edicilik

CosFace (Large Margin Cosine Loss)

ArcFace ile benzer bir mantığa sahiptir ancak aradaki farkı "açısal" olarak değil, "**cosinus**" değeri üzerinden maksimize eder. Sınıflar (kişiler) arasındaki sınırı daha net çizmek için matematiksel bir "kar payı" (margin) bırakır.

SphereFace

Yüz tanıma dünyasında "açısal marjin" (angular margin) kavramını ilk popülerleştiren modellerden biridir. Yüzleri bir hiper-küre üzerinde sınıflandırır. ArcFace'in atası sayılır ancak eğitilmesi ArcFace'e göre biraz daha zordur ve bazen kararsızlıklar yaşayabilir.

OpenFace

FaceNet'in açık kaynaklı bir versiyonu olarak doğmuştur. Özellikle mobil cihazlarda veya düşük donanımlı sistemlerde çalışabilecek kadar hafif modeller sunar. Python ve Torch tabanlıdır. Gerçek zamanlı uygulamalarda (webcam üzerinden tanıma gibi) çok popülerdir.

VGGFace ve VGGFace2

Oxford Üniversitesi Görsel Geometri Grubu (VGG) tarafından geliştirilmiştir. Çok devasa veri setleriyle (milyonlarca görsel, binlerce farklı kişi) eğitilmişlerdir. Genellikle transfer öğrenme (transfer learning) projelerinde baz model olarak kullanılırlar. ResNet veya SE-Net mimarilerini temel alırlar.

InsightFace

Bu aslında tek bir model değil, ArcFace dahil birçok modern algoritmanın ve "state-of-the-art" (SOTA) modelin toplandığı bir kütüphanedir. Günümüzde profesyonel bir yüz tanıma projesi geliştirilecekse, en güncel modelleri (örneğin Buffalo_L modelleri) barındırdığı için ilk durak burasıdır.

Zorluklar ve Etik

$y = g(x)$

Secant Lines

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$
$$f(x) = \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h}$$
$$= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h}$$
$$= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h}$$
$$= \lim_{h \rightarrow 0} h(2x + h)$$

$g(x+h) - g(x)$

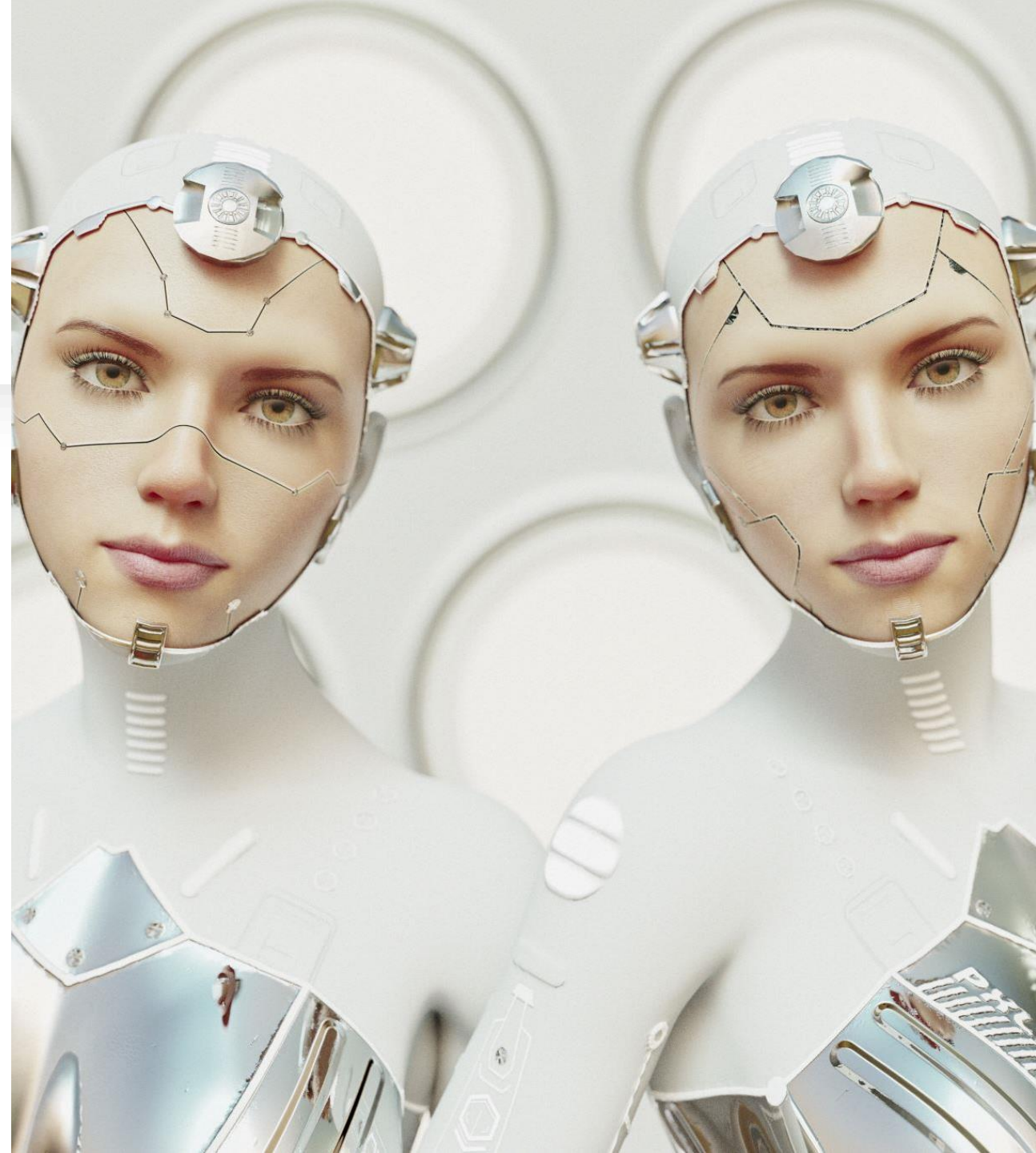
Zorluklar ve Etik

Çevresel ve Fiziksel Zorluklar

Işıklandırma (Illumination): Aşırı gölge veya parlama, yüz hatlarını gizleyebilir.

Poz Varyasyonları: Profil görüntüsü veya aşırı eğik açılar, önden çekilmiş referans fotoğrafla eşleşmeyi zorlaştırır.

Aksesuarlar ve Kozmetik: Maske, gözlük, sakal veya ağır makyajın özellikleri kapatması.



Zorluklar ve Etik

Yaşlanma ve Zaman Etkisi

Zamana Bağlı Değişim: Yüz kemik yapısı aynı kalsa da deri elastikiyeti ve doku değişir.

Çözüm: Modelin zamanla veritabanındaki embedding vektörlerini güncelleyen dinamik sistemler.



Zorluklar ve Etik

Etik ve Gizlilik Sorunları

Rıza (Consent): Kişilerin haberi olmadan biyometrik verilerinin toplanması.

Veri Güvenliği: Yüz verisi bir "parola" gibidir ama değiştirilemez. Sızdırılması durumunda geri dönüşü yoktur.

Gözetim Toplumu: Kitlesele takip sistemlerinin bireysel özgürlükler üzerindeki etkisi

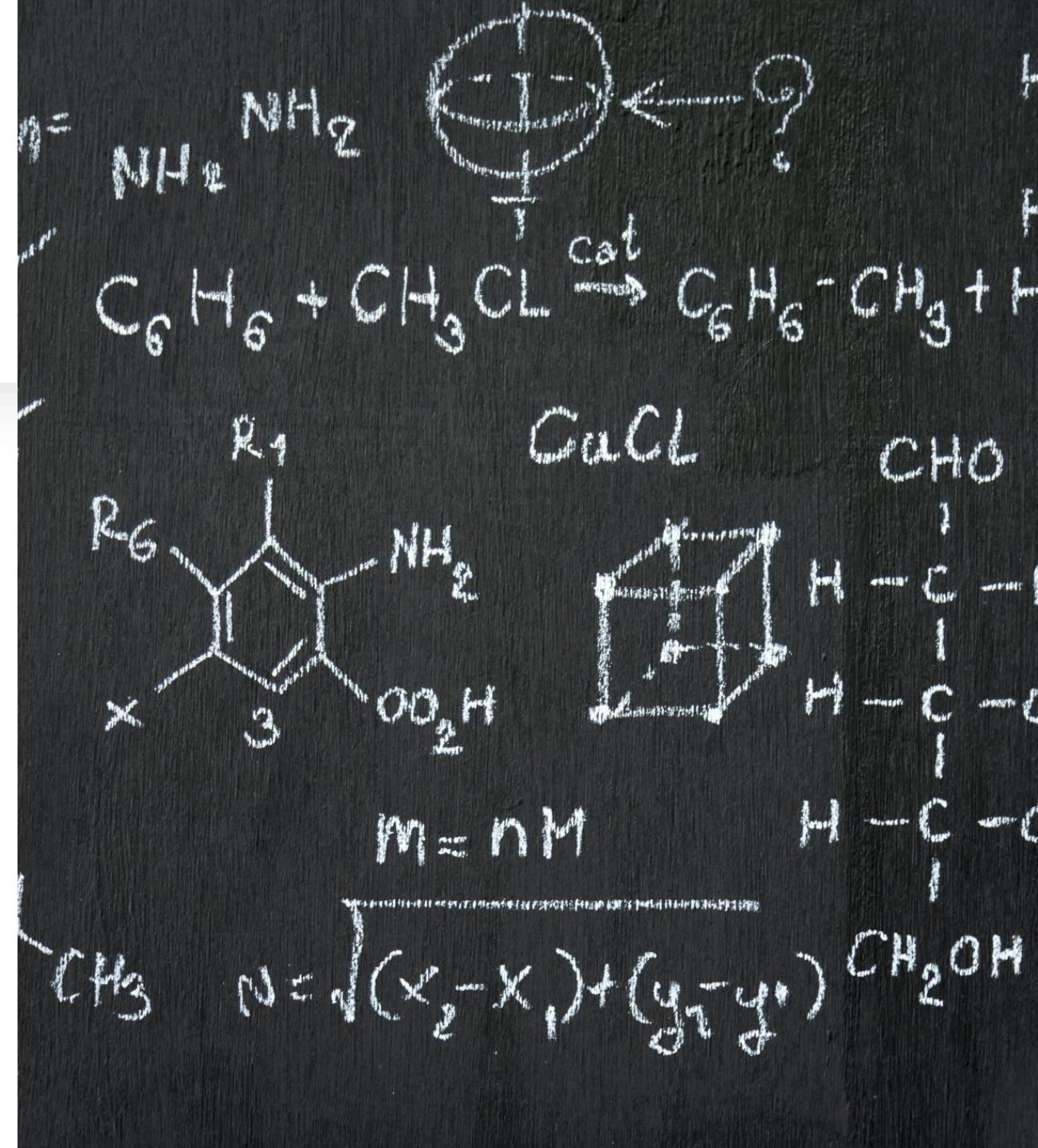


Zorluklar ve Etik

Algoritmik Önyargı (Bias)

Temsiliyet Sorunu: Eğitim setleri yeterince çeşitli değilse (farklı etnik köken, yaş veya cinsiyet), model bazı gruplarda daha düşük doğruluk oranı sergileyebilir.

Teknik Adalet: Algoritmaların herkes için eşit derecede doğru çalışması gerekliliği.





Gelecek ve Trendler

3D Face Recognition: Derinlik algılayıcı sensörlerle (LiDAR) daha güvenli sistemler.

Multimodal Sistemler: Yüz tanımanın ses veya yürüyüş analiziyle birleştirilmesi.

CIFAR-10 veri seti ile Metrik Öğrenme Denemesi

$y = g(x)$

Secant Lines

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$
$$f(x) = \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h}$$
$$= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h}$$
$$= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h}$$
$$= \lim_{h \rightarrow 0} h(2x + h)$$

$x+h$

$g(x+h) - g(x)$

$= \lim_{h \rightarrow 0} h(2x + h)$

Görüntü benzerliği araması için metrik öğrenme CIFAR-10



<http://contrib.scikit-learn.org/metric-learn/introduction.html>

```
[ ] import os

os.environ["KERAS_BACKEND"] = "tensorflow"

import random
import matplotlib.pyplot as plt
import numpy as np
import tensorflow as tf
from collections import defaultdict
from PIL import Image
from sklearn.metrics import ConfusionMatrixDisplay
import keras
from keras import layers
from keras.datasets import cifar10
```

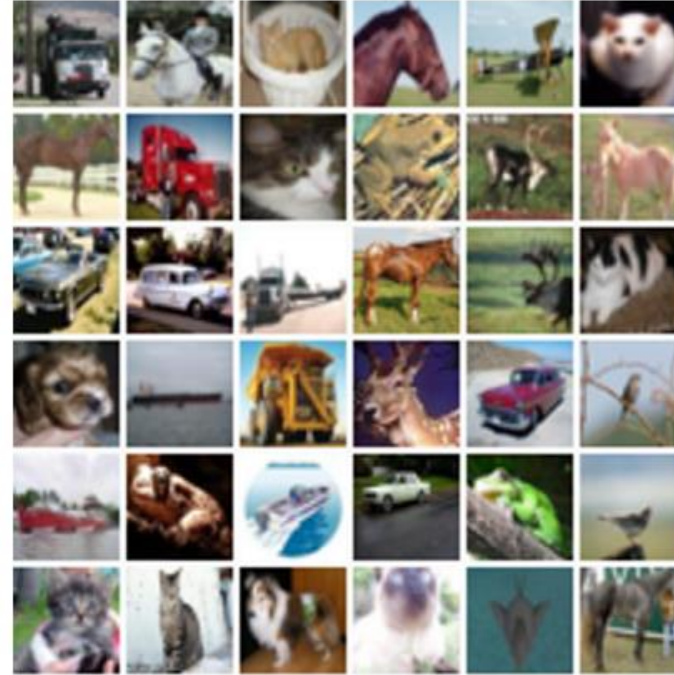
```
[ ] (x_train, y_train), (x_test, y_test) = cifar10.load_data()

x_train = x_train.astype("float32") / 255.0
y_train = np.squeeze(y_train)
x_test = x_test.astype("float32") / 255.0
y_test = np.squeeze(y_test)
```

Downloading data from <https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz>
170498071/170498071 ————— 11s 0us/step

```
height_width = 32
```

```
def show_collage(examples):  
    box_size = height_width + 2  
    num_rows, num_cols = examples.shape[:2]  
  
    collage = Image.new(  
        mode="RGB",  
        size=(num_cols * box_size, num_rows * box_size),  
        color=(250, 250, 250),  
    )  
    for row_idx in range(num_rows):  
        for col_idx in range(num_cols):  
            array = (np.array(examples[row_idx, col_idx]) * 255).astype(np.uint8)  
            collage.paste(  
                Image.fromarray(array), (col_idx * box_size, row_idx * box_size)  
            )  
  
    # Double size for visualisation.  
    collage = collage.resize((2 * num_cols * box_size, 2 * num_rows * box_size))  
    return collage  
  
# Show a collage of 5x5 random images.  
sample_idx = np.random.randint(0, 50000, size=(6, 6))  
inputs = x_train[sample_idx]  
show_collage(inputs)
```



Yukarıdaki görseller için tanımlanmış olan cifar10 indeks değerleri (Örnek 3=Kedi)

```
outputs = y_train[sample_idxs]
print (outputs)
```

```
[[9 7 3 7 0 3]
 [7 9 3 6 4 7]
 [1 1 9 7 4 3]
 [5 8 9 4 1 2]
 [8 6 8 1 6 2]
 [3 3 5 3 0 7]]
```

Metrik öğrenmede eğitim verisi doğrudan etiketli çiftler yerine, rastgele seçilen bir çapa (anchor) ve aynı sınıftan bir pozitif örnek arasındaki benzerlik ilişkisi üzerinden kurgulanır. Bu süreci yürütmek için sınıfları ilgili örneklerle eşleştiren bir dizin yapısı oluşturulur ve eğitim sırasında veriler bu yapıdan örneklenerek kullanılır.

```
class_idx_to_train_idxs = defaultdict(list)
for y_train_idx, y in enumerate(y_train):
    class_idx_to_train_idxs[y].append(y_train_idx)

class_idx_to_test_idxs = defaultdict(list)
for y_test_idx, y in enumerate(y_test):
    class_idx_to_test_idxs[y].append(y_test_idx)
```

Kedi görsellerinin olduğu indeksler için 3 sayısı (ilk 5 tanesini göstereyin)

```
print(class_idx_to_train_idxs[3][:5])
```

```
[9, 17, 21, 26, 33]
```


Eğitim sürecinde her sınıftan birer adet olmak üzere oluşturulan (çapa, pozitif) çiftleri, aynı kümedeki diğer örneklerden uzaklaştırılıp birbirine yaklaştırılmaya çalışılır. Bu yöntemde veri paketinin (batch) boyutu doğrudan sınıf sayısına bağlıdır; örneğin CIFAR-10 veri kümesi için bu sayı 10 olarak belirlenir.

```
num_classes = 10

class AnchorPositivePairs(keras.utils.Sequence):
    def __init__(self, num_batches):
        super().__init__()
        self._num_batches = num_batches

    def __len__(self):
        return self._num_batches

    def __getitem__(self, _idx):
        x = np.empty((2, num_classes, height_width, height_width, 3), dtype=np.float32)
        for class_idx in range(num_classes):
            examples_for_class = class_idx_to_train_idxs[class_idx]
            anchor_idx = random.choice(examples_for_class)
            positive_idx = random.choice(examples_for_class)
            while positive_idx == anchor_idx:
                positive_idx = random.choice(examples_for_class)
            x[0, class_idx] = x_train[anchor_idx]
            x[1, class_idx] = x_train[positive_idx]
        return x
```

Oluşturulan veri paketini bir kolaj yardımıyla görselleştirmek mümkündür. Bu görselde üst satır, on farklı sınıftan rastgele seçilmiş çapa (anchor) örneklerini, alt satır ise bunlara karşılık gelen on pozitif örneği gösterir. Uçak, Araba, Kuş, Kedi, Geyik, Köpek, Kurbaa, At, Gemi, Kamyon

```
inputs = next(iter(AnchorPositivePairs(num_batches=1)))

show_collage(inputs)
```



Tanımlanan bu modelin eğitim adımında (train_step), öncelikle çapa ve pozitif örneklerin sayısal vektör karşılıkları (embedding) oluşturulur. Ardından, bu vektörlerin birbirleriyle olan nokta çarpımları (dot product) hesaplanarak, benzerlik skorlarını belirleyen bir softmax katmanı için girdi (logits) olarak kullanılır.

```
class EmbeddingModel(keras.Model):
    def train_step(self, data):
        # Not: Açık soruna yönelik geçici çözüm kaldırılacaktır.
        if isinstance(data, tuple):
            data = data[0]
        anchors, positives = data[0], data[1]

        with tf.GradientTape() as tape:
            # Hem çapraz hem de pozitif değerleri model üzerinden geçirir.

            anchor_embeddings = self(anchors, training=True)
            positive_embeddings = self(positives, training=True)

            # Çapa noktaları ve pozitif noktalar arasındaki kosinüs benzerliğini hesaplar.
            # Normalleştirildikleri için bu sadece ikili nokta çarpımlarıdır.

            benzerlik = keras.ops.einsum(
                "ae,pe->ap", anchor_embeddings, positive_embeddings
            )

            # Bunları logit olarak kullanmayı planladığımız için, bunları bir katsayıyla ölçeklendiriyoruz.
            # Bu değer normalde bir hiper parametre olarak seçilir. temperature olarak geçiyor literatürde.

            katsayi = 0.1
            benzerlik /= katsayi

            # Çapa/pozitif çiftlere karşılık gelen ana diyagonal değerlerin yüksek olması isteniyor.
            # Bu kayıp, çapa/pozitif çiftler için gömülü vektörleri birbirine yaklaştıracak ve
            # diğer tüm çiftleri birbirinden uzaklaştıracaktır.

            sparse_labels = keras.ops.arange(num_classes)
            loss = self.compute_loss(y=sparse_labels, y_pred= benzerlik)

        # Eğitimleri hesaplar ve optimizasyon algoritması aracılığıyla uygular.

        gradients = tape.gradient(loss, self.trainable_variables)
        self.optimizer.apply_gradients(zip(gradients, self.trainable_variables))

        # Güncellemeleri yapar ve ölçüm değerlerini (özellikle kayıp değeriyle ilgili olanı) döndürür.
        for metric in self.metrics:

            if metric.name == "loss":
                metric.update_state(loss)
            else:
                metric.update_state(sparse_labels, benzerlik)

        return {m.name: m.result() for m in self.metrics}
```

Görüntüleri vektör uzayına aktaran bu model; evrişimli katmanlar (CNN), global havuzlama ve doğrusal bir projeksiyon katmanından oluşur.

Benzerlik hesaplamasını kolaylaştırmak için vektörler normalize edilir ve model yapısı basitlik adına küçük tutulmuştur.

```
▶ inputs = layers.Input(shape=(height_width, height_width, 3))
x = layers.Conv2D(filters=32, kernel_size=3, strides=2, activation="relu")(inputs)
x = layers.Conv2D(filters=64, kernel_size=3, strides=2, activation="relu")(x)
x = layers.Conv2D(filters=128, kernel_size=3, strides=2, activation="relu")(x)
x = layers.GlobalAveragePooling2D()(x)

embeddings = layers.Dense(units=64, activation=None)(x)
embeddings = layers.UnitNormalization()(embeddings)

model = EmbeddingModel(inputs, embeddings)
```



Son olarak eğitimi başlatıyoruz. Google Colab (CPU 5 dakika) GPU örneğinde 1 dakika sürüyor.

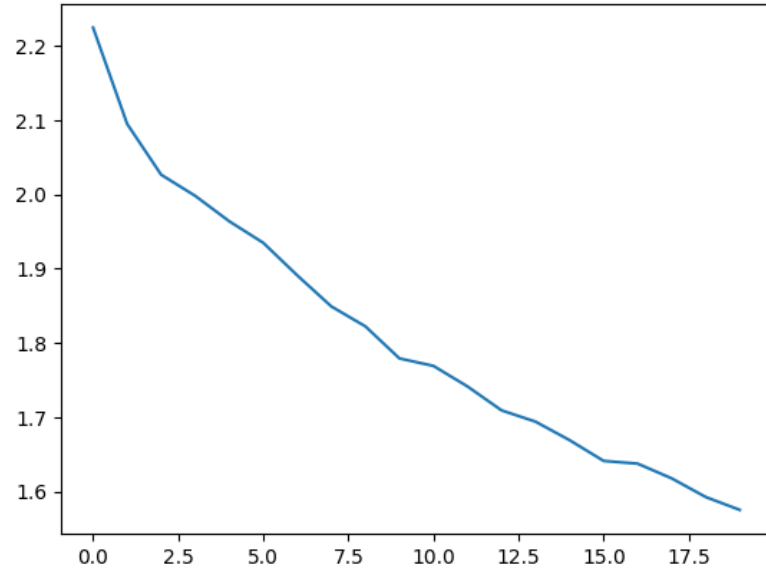
```
▶ model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-3),
    loss=keras.losses.SparseCategoricalCrossentropy(from_logits=True),
)

history = model.fit(AnchorPositivePairs(num_batches=1000), epochs=20)

plt.plot(history.history["loss"])
plt.show()
```

```
... Epoch 1/20
1000/1000 ————— 7s 3ms/step - loss: 2.2245
Epoch 2/20
1000/1000 ————— 8s 3ms/step - loss: 2.0949
Epoch 3/20
1000/1000 ————— 3s 3ms/step - loss: 2.0264
Epoch 4/20
1000/1000 ————— 3s 3ms/step - loss: 1.9980
Epoch 5/20
1000/1000 ————— 8s 6ms/step - loss: 1.9637
Epoch 6/20
1000/1000 ————— 3s 3ms/step - loss: 1.9347
Epoch 7/20
1000/1000 ————— 3s 3ms/step - loss: 1.8908
Epoch 8/20
1000/1000 ————— 3s 3ms/step - loss: 1.8490
Epoch 9/20
1000/1000 ————— 3s 3ms/step - loss: 1.8222
Epoch 10/20
1000/1000 ————— 4s 4ms/step - loss: 1.7791
Epoch 11/20
1000/1000 ————— 4s 4ms/step - loss: 1.7690
Epoch 12/20
1000/1000 ————— 3s 3ms/step - loss: 1.7413
Epoch 13/20
```

Epoch 13/20
1000/1000 — 3s 3ms/step - loss: 1.7091
Epoch 14/20
1000/1000 — 3s 3ms/step - loss: 1.6939
Epoch 15/20
1000/1000 — 3s 3ms/step - loss: 1.6690
Epoch 16/20
1000/1000 — 3s 3ms/step - loss: 1.6411
Epoch 17/20
1000/1000 — 3s 3ms/step - loss: 1.6374
Epoch 18/20
1000/1000 — 3s 3ms/step - loss: 1.6175
Epoch 19/20
1000/1000 — 3s 3ms/step - loss: 1.5924
Epoch 20/20
1000/1000 — 3s 3ms/step - loss: 1.5751



Öncelikle test kümesini gömüyoruz ve tüm yakın komşuları hesaplıyoruz. Gömme işlemlerinin birim uzunlukta olduğunu hatırlarsak, nokta çarpımları yoluyla kosinüs benzerliğini hesaplayabiliriz.

```
near_neighbours_per_example = 10

embeddings = model.predict(x_test)
gram_matrix = np.einsum("ae,be->ab", embeddings, embeddings)
near_neighbours = np.argsort(gram_matrix.T)[: , -(near_neighbours_per_example + 1) :]
```

313/313 — 1s 3ms/step

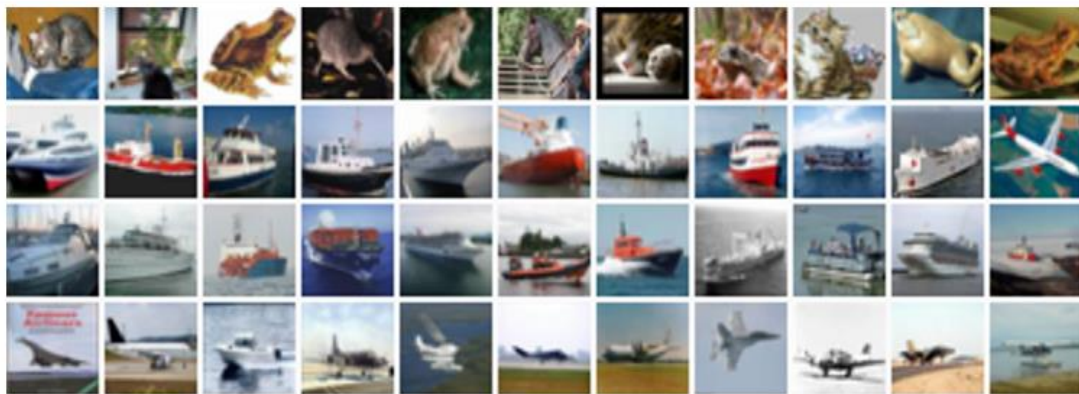
Gömülü vektörlerin (embeddings) başarısını görsel olarak denetlemek için, rastgele seçilen 4 örnek ve bunların en yakın komşularından oluşan bir kolaj oluşturuldu. Bu görselde ilk sütun hedef görüntüyü, sonraki 10 sütun ise benzerlik sırasına göre dizilmiş en yakın komşuları temsil eder.

```
num_collage_examples = 4

examples = np.empty(
    (
        num_collage_examples,
        near_neighbours_per_example + 1,
        height_width,
        height_width,
        3,
    ),
    dtype=np.float32,
)

for row_idx in range(num_collage_examples):
    examples[row_idx, 0] = x_test[row_idx]
    anchor_near_neighbours = reversed(near_neighbours[row_idx][: -1])
    for col_idx, nn_idx in enumerate(anchor_near_neighbours):
        examples[row_idx, col_idx + 1] = x_test[nn_idx]

show_collage(examples)
```



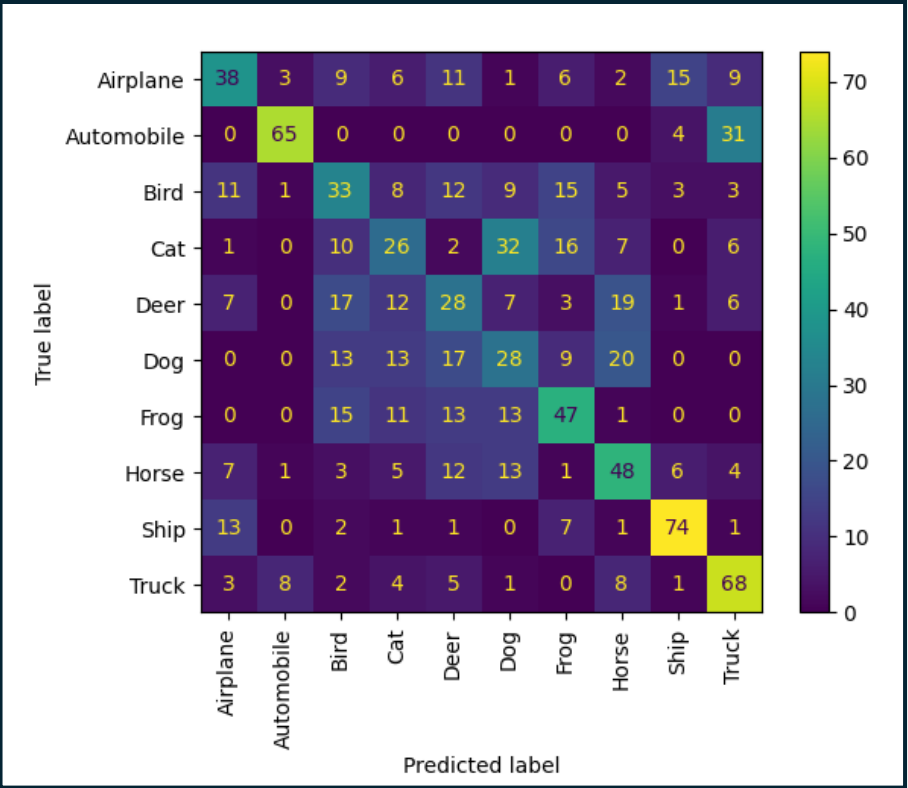
Performans analizi için her sınıftan on örnek seçilip yakın komşularının aynı sınıfa ait olup olmadığına bakılarak bir karışıklık matrisi (confusion matrix) oluşturulur. Gözlemler sonucunda hayvan ve araç sınıflarının kendi içlerinde yüksek doğruluk sergilediği, hataların ise genellikle benzer kategoriler (hayvanların hayvanlarla, araçların araçlarla) arasında yaşandığı görülür.

```
confusion_matrix = np.zeros((num_classes, num_classes))

# For each class.
for class_idx in range(num_classes):
    # Consider 10 examples.
    example_idx = class_idx_to_test_idxs[class_idx][:10]
    for y_test_idx in example_idx:
        # And count the classes of its near neighbours.
        for nn_idx in near_neighbours[y_test_idx][:1]:
            nn_class_idx = y_test[nn_idx]
            confusion_matrix[class_idx, nn_class_idx] += 1

# Display a confusion matrix.
labels = [
    "Airplane",
    "Automobile",
    "Bird",
    "Cat",
    "Deer",
    "Dog",
    "Frog",
    "Horse",
    "Ship",
    "Truck",
]

disp = ConfusionMatrixDisplay(confusion_matrix=confusion_matrix, display_labels=labels)
disp.plot(include_values=True, cmap="viridis", ax=None, xticks_rotation="vertical")
plt.show()
```



```
from google.colab import files
```

```
# Dosya Yükle
```

```
print("Lütfen denemek istediğiniz resimleri seçin:")
```

```
uploaded = files.upload()
```

```
# Sınıf isimleri
```

```
labels = [
```

```
    "Airplane (Uçak)", "Automobile (Otomobil)", "Bird (Kuş)", "Cat (Kedi)",
```

```
    "Deer (Geyik)", "Dog (Köpek)", "Frog (Kurbağa)", "Horse (At)",
```

```
    "Ship (Gemi)", "Truck (Kamyon)"
```

```
]
```

```
# 2. Resim İşleme Fonksiyonu
```

```
def process_and_predict(img_path, model, test_embeddings, y_test):
```

```
    # Resmi aç ve 32x32 RGB yap
```

```
    img = Image.open(img_path).convert('RGB').resize((32, 32))
```

```
    img_array = np.array(img).astype('float32') / 255.0
```

```
    img_tensor = np.expand_dims(img_array, axis=0)
```

```
    # Modelden embedding al
```

```
    target_embedding = model.predict(img_tensor, verbose=0)
```

```
    # Benzerlik hesapla (Dot Product)
```

```
    similarities = np.dot(target_embedding, test_embeddings.T)
```

```
    # En yakın komşuyu bul
```

```
    closest_idx = np.argmax(similarities)
```

```
    predicted_class = y_test[closest_idx]
```

```
    return img, labels[predicted_class]
```

```
# 3. Görselleştirme ve Döngü
```

```
# Test setindeki embeddingleri önceden hazırla (hız için)
```

```
test_embeddings = model.predict(x_test, verbose=0)
```

```
plt.figure(figsize=(10, 10))
```

```
num_uploaded_items = len(uploaded.items())
```

```
num_cols = int(np.ceil(num_uploaded_items / 2.0)) if num_uploaded_items > 0 else 1
```

```
num_rows = 2 if num_uploaded_items > 2 else 1 # Eğer 2'den fazla resim varsa 2 satır, yoksa 1
```

```
for i, (filename, content) in enumerate(uploaded.items()):
```

```
    # Tahmin yürüt
```

```
    pil_img, result_text = process_and_predict(filename, model, test_embeddings, y_test)
```

```
    # Ekrana çizdir
```

```
    plt.subplot(num_rows, num_cols, i + 1)
```

```
    plt.imshow(pil_img)
```

```
    plt.title(f"Tahmin: {result_text}\n({filename})")
```

```
    plt.axis('off')
```

```
plt.tight_layout()
```

```
plt.show()
```

Lütfen denemek istediğiniz resimleri seçin:

Dosyaları Seç

Dosya seçilmedi

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving res1.jpg to res1 (13).jpg

Saving res2.jpg to res2 (13).jpg

Saving res3.jpg to res3 (13).jpg

Saving res4.jpg to res4 (15).jpg

Saving res5.jpg to res5 (12).jpg

Saving res6.jpg to res6 (12).jpg

Saving res7.jpg to res7 (6).jpg

Saving res8.jpg to res8 (6).jpg

Saving res9.jpg to res9 (2).jpg

Tahmin: Cat (Kedi)
(res1 (13).jpg)



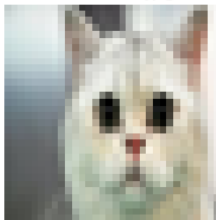
Tahmin: Horse (At)
(res2 (13).jpg)



Tahmin: Automobile (Otomobil)
(res3 (13).jpg)



Tahmin: Dog (Köpek)
(res4 (15).jpg)



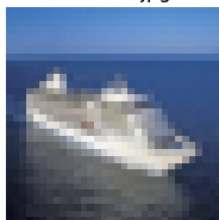
Tahmin: Airplane (Uçak)
(res5 (12).jpg)



Tahmin: Truck (Kamyon)
(res6 (12).jpg)



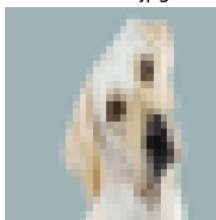
Tahmin: Ship (Gemi)
(res7 (6).jpg)



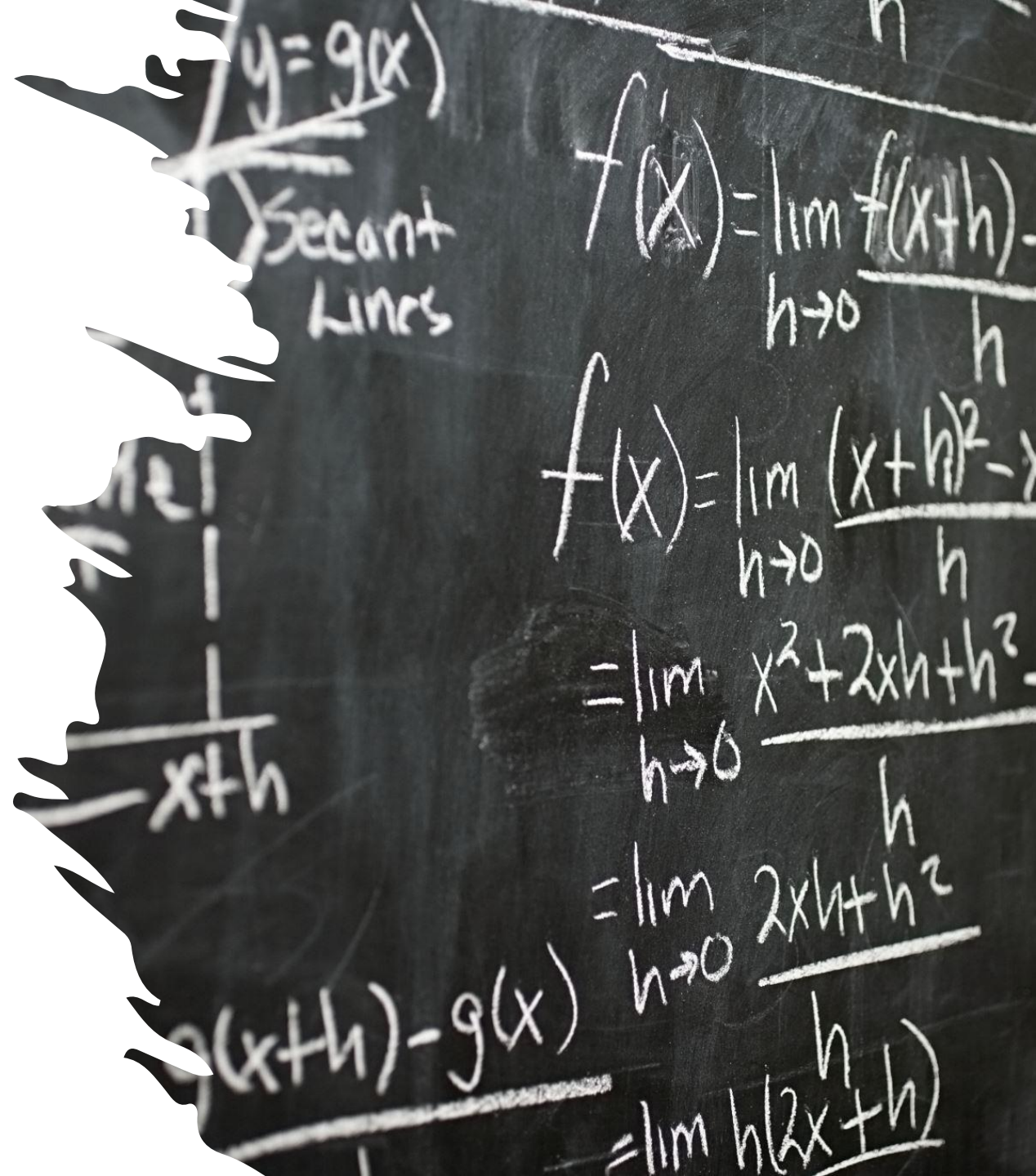
Tahmin: Deer (Geyik)
(res8 (6).jpg)



Tahmin: Dog (Köpek)
(res9 (2).jpg)



72 görsel ve Triplet Loss Yöntemi ile Face Recognition Denemesi



Triplet Loss deneme

```
import numpy as np
import pandas as pd
import tensorflow as tf
import cv2
import matplotlib.pyplot as plt
import os
from sklearn.preprocessing import LabelEncoder
import zipfile
from sklearn.decomposition import PCA
from sklearn.neighbors import KNeighborsClassifier
from google.colab import files
```

```
zip_path = '/content/drive/MyDrive/archive.zip'
unzip_dir = '/content/archive/'
os.makedirs(unzip_dir, exist_ok=True)
```

```
with zipfile.ZipFile(zip_path, 'r') as zip_ref:
    zip_ref.extractall(unzip_dir)
```

```
▶ for root, dirs, files in os.walk('/content/archive/'):
    for name in files:
        print(os.path.join(root, name))
    for name in dirs:
        print(os.path.join(root, name))
```

... [Gizli çıkışı göster](#)

```
!wget https://raw.githubusercontent.com/opencv/opencv/master/data/haarcascades/haarcascade_frontalface_default.xml -O haar.xml
```

```
--2026-04-25 15:47:13-- https://raw.githubusercontent.com/opencv/opencv/master/data/haarcascades/haarcascade_frontalface_default.xml
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.108.133, 185.199.109.133, 185.199.111.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.108.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 930127 (908K) [text/plain]
Saving to: 'haar.xml'
```

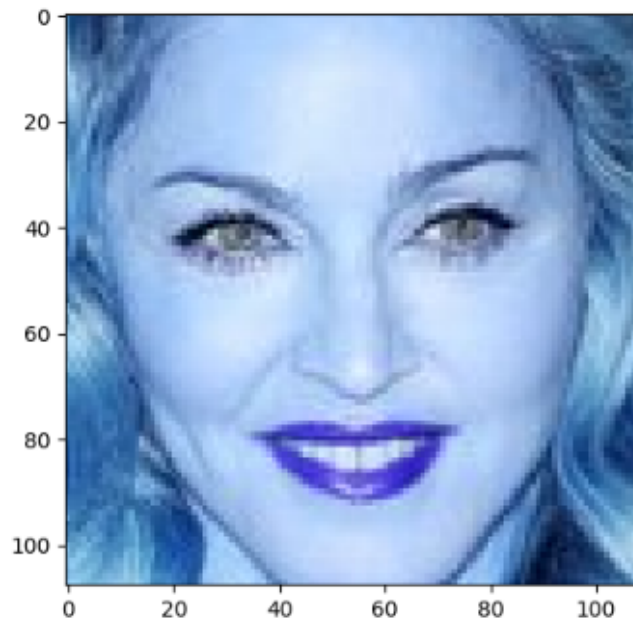
```
haar.xml          100%[=====>] 908.33K  ---KB/s   in 0.05s
```

```
2026-04-25 15:47:13 (19.4 MB/s) - 'haar.xml' saved [930127/930127]
```

```
haar_cascade_path = '/content/haar.xml'
face_cascade = cv2.CascadeClassifier(haar_cascade_path)
image_path = "/content/archive/data/train/madonna/httpiamediaimdbcomimagesMMVBMTANDQNTAxNDVeQTJeQWpwZBbwUMDIMjQOTYVUXCRAL.jpg.jpg"
f = cv2.imread(image_path)

faces = face_cascade.detectMultiScale(f,1.3,5)
print(faces)
for x,y,w,h in faces:
    plt.imshow(f[y:y+h, x:x+w])
```

```
[[ 50  50 108 108]]
```



```
dirs = "/content/archive/data/train/"
img_size = 60
data = []
for name in os.listdir(dirs):
    for f in os.listdir(dirs+name):
        f = cv2.imread(os.path.join(dirs+name, f))
        faces = face_cascade.detectMultiScale(f,1.3,5)
        for x,y,w,h in faces:
            img = f[y:y+h, x:x+w]
            img = cv2.resize(img, (img_size,img_size))
            data.append((img, name))

df = pd.DataFrame(data, columns=["image", "name"])
print("Length:",len(df))
```

Length: 75

df.head()

	image	name
0	[[[250, 249, 251], [254, 253, 254], [252, 252, ...	jerry_seinfeld
1	[[[38, 49, 80], [66, 83, 124], [90, 113, 162], ...	jerry_seinfeld
2	[[[250, 250, 250], [250, 250, 250], [250, 250, ...	jerry_seinfeld
3	[[[237, 232, 223], [237, 232, 223], [236, 231, ...	jerry_seinfeld
4	[[[51, 65, 54], [37, 52, 44], [44, 61, 57], [9...	jerry_seinfeld

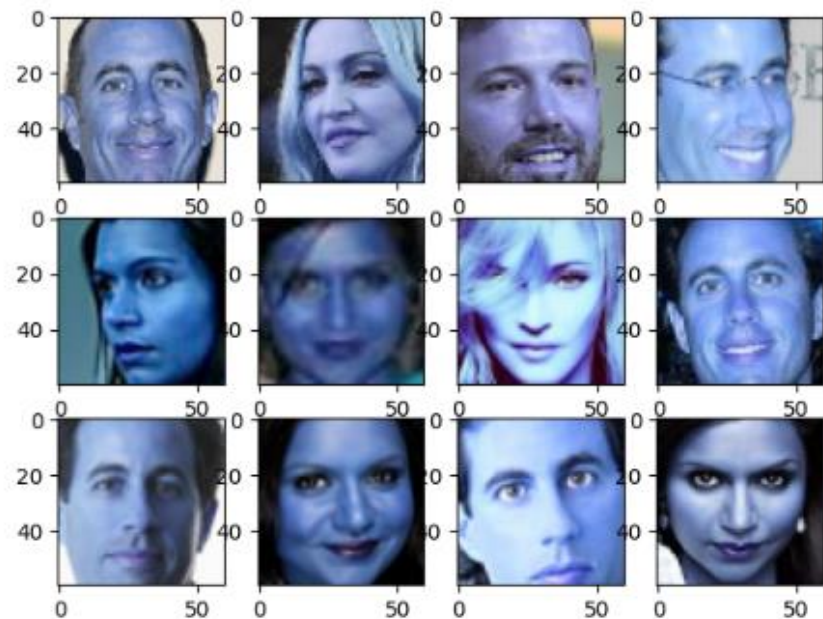
Sonraki adımlar: [df ile kod oluşturun](#)

[New interactive sheet](#)

```

for i in range(12):
    row = df.iloc[np.random.randint(0, len(df)), :]
    plt.subplot(3,4,i+1)
    plt.imshow(row['image'])

```



```

dirs = "/content/archive/data/val/"

data = []
for name in os.listdir(dirs):
    for f in os.listdir(dirs+name):
        f = cv2.imread(os.path.join(dirs+name, f))
        faces = face_cascade.detectMultiScale(f,1.3,5)
        for x,y,w,h in faces:
            img = f[y:y+h, x:x+w]
            img = cv2.resize(img, (img_size,img_size))
            data.append((img, name))

df_test = pd.DataFrame(data, columns=["image", "name"])
print("Test size: ", len(df_test))

```

Test size: 24


```
df_test.head()
```

	image	name
0	[[[56, 48, 49], [53, 48, 50], [52, 51, 53], [5...	jerry_seinfeld
1	[[[33, 30, 23], [30, 27, 22], [26, 24, 20], [2...	jerry_seinfeld
2	[[[102, 131, 171], [83, 112, 148], [51, 54, 54...	jerry_seinfeld
3	[[[5, 14, 4], [5, 14, 4], [5, 17, 5], [5, 21, ...	jerry_seinfeld
4	[[[24, 36, 78], [23, 35, 78], [23, 36, 76], [1...	jerry_seinfeld



Sonraki adımlar:

[df_test ile kod oluşturun](#)[New interactive sheet](#)

```
le = LabelEncoder()  
le.fit(df["name"].values)
```

LabelEncoder

LabelEncoder()

```
x_train = list(df['image'].values)
x_train = np.array(x_train)
x_train = x_train/255
print(x_train.shape)

y_train = le.transform(df["name"].values)
print(y_train.shape)
```

```
(75, 60, 60, 3)
(75,)
```

```
x_test = list(df_test['image'].values)
x_test = np.array(x_test)
x_test = x_test/255
print(x_test.shape)

y_test = le.transform(df_test["name"].values)
print(y_test.shape)
```

```
(24, 60, 60, 3)
(24,)
```

```
people_num = len(np.unique(y_train))
people_num
```

```
5
```

```
def triplet_loss(y_true, y_pred, alpha = 0.2):
    total_lenght = y_pred.shape.as_list()[-1]
    anchor, positive, negative = y_pred[:, :int(1/3*total_lenght)], y_pred[:, int(1/3*total_lenght):int(2/3*total_lenght)], y_pred[:, int(2/3*total_lenght):]

    pos_dist = tf.reduce_sum(tf.square(anchor - positive), axis=-1)
    neg_dist = tf.reduce_sum(tf.square(anchor - negative), axis=-1)
    basic_loss = pos_dist - neg_dist + alpha
    loss = tf.reduce_sum(tf.maximum(basic_loss, 0.0))
    return loss
```

```
def generate_triplets(x, y, num_same, num_diff):
    anchor_images = np.array([]).reshape((-1,) + x.shape[1:])
    same_images = np.array([]).reshape((-1,) + x.shape[1:])
    diff_images = np.array([]).reshape((-1,) + x.shape[1:])

    for i in range(len(y)):
        point = y[i]
        anchor = x[i]

        same_pairs = np.where(y == point)[0]
        same_pairs = np.delete(same_pairs, np.where(same_pairs == i))
        diff_pairs = np.where(y != point)[0]

        same = x[np.random.choice(same_pairs, num_same)]
        diff = x[np.random.choice(diff_pairs, num_diff)]

        anchor_images = np.concatenate((anchor_images, np.tile(anchor, (num_same * num_diff, 1, 1, 1))), axis = 0)

        for s in same:
            same_images = np.concatenate((same_images, np.tile(s, (num_same, 1, 1, 1))), axis = 0)
            diff_images = np.concatenate((diff_images, np.tile(diff, (num_diff, 1, 1, 1))), axis = 0)

    return anchor_images, same_images, diff_images
```

```
anchor_images, same_images, diff_images = generate_triplets(x_train, y_train, num_same=10, num_diff=10)
print(anchor_images.shape, same_images.shape, diff_images.shape)
```

```
(7500, 60, 60, 3) (7500, 60, 60, 3) (7500, 60, 60, 3)
```

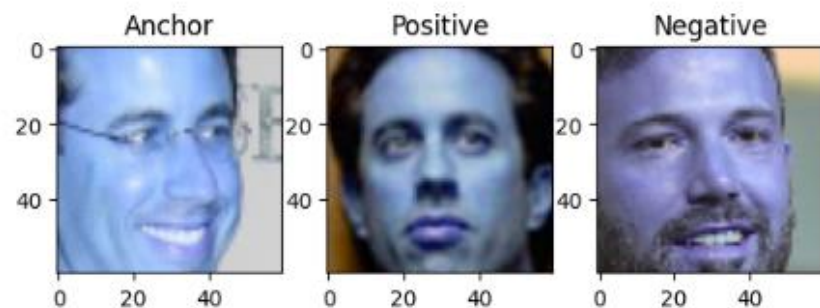
```

idx = 100
plt.subplot(1,3,1)
plt.imshow(anchor_images[idx])
plt.title('Anchor')

plt.subplot(1,3,2)
plt.imshow(same_images[idx])
plt.title('Positive')

plt.subplot(1,3,3)
plt.imshow(diff_images[idx])
plt.title('Negative')
print()

```



```

def get_model():
    model = tf.keras.Sequential()
    model.add(tf.keras.layers.Conv2D(64, kernel_size=3, strides=2, padding='same', input_shape=(img_size,img_size,3), activation='relu'))
    model.add(tf.keras.layers.Conv2D(128, kernel_size=3, strides=2, padding='same', activation='relu'))
    model.add(tf.keras.layers.Conv2D(64, kernel_size=3, strides=2, padding='same', activation='relu'))
    model.add(tf.keras.layers.Conv2D(64, kernel_size=1, strides=2, padding='same', activation='relu'))
    model.add(tf.keras.layers.Flatten())

    model.add(tf.keras.layers.Dense(256, activation='relu'))
    model.add(tf.keras.layers.Dropout(0.2))
    model.add(tf.keras.layers.Dense(128))

    model.summary()
    return model

```

```

anchor_input = tf.keras.layers.Input((img_size, img_size, 3), name='anchor_input')
positive_input = tf.keras.layers.Input((img_size, img_size, 3), name='positive_input')
negative_input = tf.keras.layers.Input((img_size, img_size, 3), name='negative_input')

shared_dnn = get_model()
encoded_anchor = shared_dnn(anchor_input)
encoded_positive = shared_dnn(positive_input)
encoded_negative = shared_dnn(negative_input)
merged_vector = tf.keras.layers.concatenate([encoded_anchor, encoded_positive, encoded_negative], axis=-1, name='merged_layer')

model = tf.keras.Model(inputs=[anchor_input, positive_input, negative_input], outputs=merged_vector)
model.summary()
model.compile(loss=triplet_loss, optimizer="adam")

```

... Model: "sequential_3"

Layer (type)	Output Shape	Param #
conv2d_12 (Conv2D)	(None, 30, 30, 64)	1,792
conv2d_13 (Conv2D)	(None, 15, 15, 128)	73,856
conv2d_14 (Conv2D)	(None, 8, 8, 64)	73,792
conv2d_15 (Conv2D)	(None, 4, 4, 64)	4,160
flatten_3 (Flatten)	(None, 1024)	0
dense_6 (Dense)	(None, 256)	262,400
dropout_3 (Dropout)	(None, 256)	0
dense_7 (Dense)	(None, 128)	32,896

Total params: 448,896 (1.71 MB)

Trainable params: 448,896 (1.71 MB)

Non-trainable params: 0 (0.00 B)

Model: "functional_37"

Layer (type)	Output Shape	Param #	Connected to
anchor_input (InputLayer)	(None, 60, 60, 3)	0	-
positive_input (InputLayer)	(None, 60, 60, 3)	0	-
negative_input (InputLayer)	(None, 60, 60, 3)	0	-
sequential_3 (Sequential)	(None, 128)	448,896	anchor_input[0][...] positive_input[0][...] negative_input[0][...]
merged_layer (Concatenate)	(None, 384)	0	sequential_3[0][...] sequential_3[1][...] sequential_3[2][...]

Total params: 448,896 (1.71 MB)

Trainable params: 448,896 (1.71 MB)

Non-trainable params: 0 (0.00 B)



```
Y_dummy = np.empty((anchor_images.shape[0],1))  
model.fit([anchor_images,same_images,diff_images],y=Y_dummy, batch_size=128, epochs=10)
```

```
... /usr/local/lib/python3.12/dist-packages/keras/src/trainers/data_adapters/array_slicing.py:63: RuntimeWarning: overflow encountered in cast  
    return x.astype(dtype)
```

```
Epoch 1/10  
59/59 ━━━━━━━━━━━ 12s 114ms/step - loss: 7.3708  
Epoch 2/10  
59/59 ━━━━━━━━━━━ 2s 36ms/step - loss: 0.3073  
Epoch 3/10  
59/59 ━━━━━━━━━━━ 2s 35ms/step - loss: 0.1776  
Epoch 4/10  
59/59 ━━━━━━━━━━━ 2s 35ms/step - loss: 0.1049  
Epoch 5/10  
59/59 ━━━━━━━━━━━ 2s 34ms/step - loss: 0.1596  
Epoch 6/10  
59/59 ━━━━━━━━━━━ 2s 37ms/step - loss: 0.0982  
Epoch 7/10  
59/59 ━━━━━━━━━━━ 3s 38ms/step - loss: 0.1678  
Epoch 8/10  
59/59 ━━━━━━━━━━━ 2s 35ms/step - loss: 0.2128  
Epoch 9/10  
59/59 ━━━━━━━━━━━ 2s 34ms/step - loss: 0.0666  
Epoch 10/10  
59/59 ━━━━━━━━━━━ 2s 35ms/step - loss: 0.0320  
<keras.src.callbacks.history.History at 0x7debf81088c0>
```

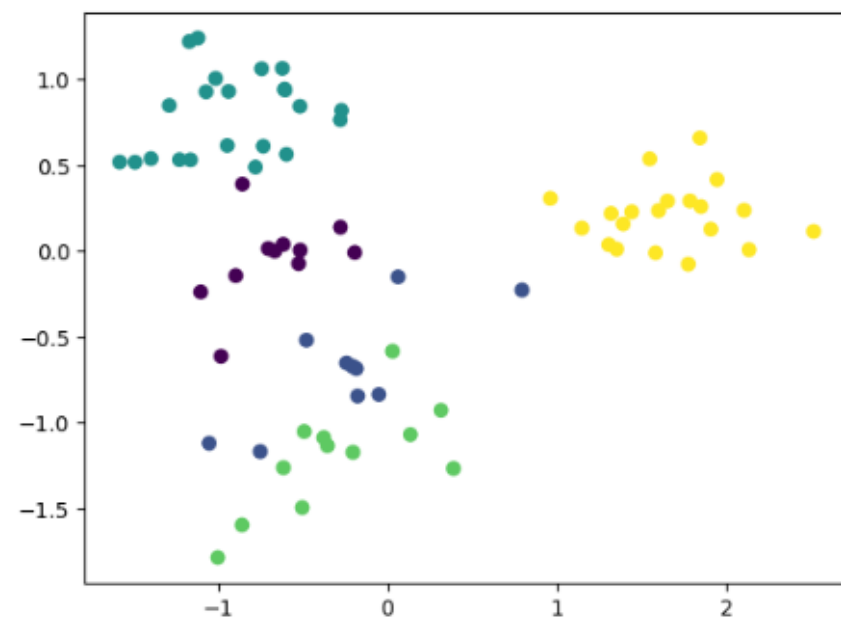
```
anchor_model = tf.keras.Model(inputs = anchor_input, outputs=encoded_anchor)
```

```
pred = anchor_model.predict(x_train)  
pred.shape
```

```
3/3 ━━━━━━━━━━━ 1s 213ms/step  
(75, 128)
```

```
pca = PCA(n_components=2)
pred_pca = pca.fit_transform(pred)
plt.scatter(pred_pca[:,0], pred_pca[:,1], c=y_train)
```

<matplotlib.collections.PathCollection at 0x7debea2ea390>



```
def encode_image(model, img):
    encode = model.predict(img.reshape((1,) + img.shape))
    return encode
```

```
pred_x_train = anchor_model.predict(x_train)
```

3/3 ————— 0s 14ms/step

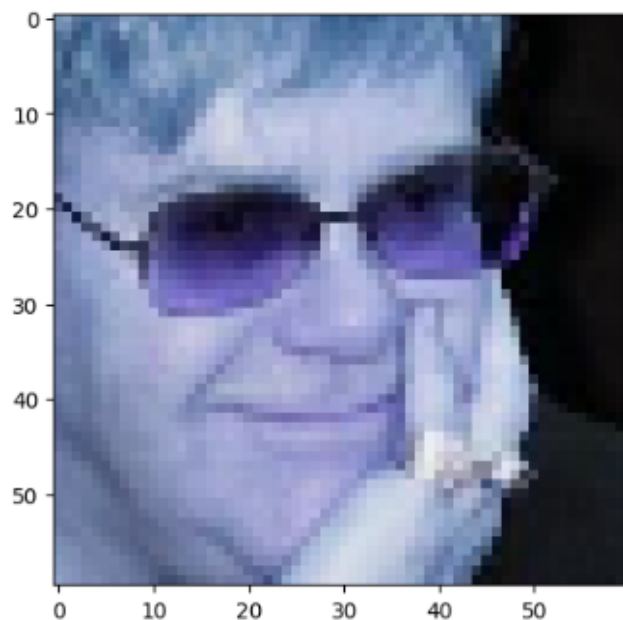
```
neigh = KNeighborsClassifier(n_neighbors=7)
neigh.fit(pred_x_train, y_train)
```

▼ KNeighborsClassifier ⓘ ⓘ
KNeighborsClassifier(n_neighbors=7)

```
idx = 8
img = x_test[idx]
plt.imshow(img)

enc = encode_image(anchor_model, img)
pred = neigh.predict(enc)
print("Predicted name:", le.inverse_transform(pred))
print("Actual pred: ", le.inverse_transform(y_test[idx:idx+1]))
```

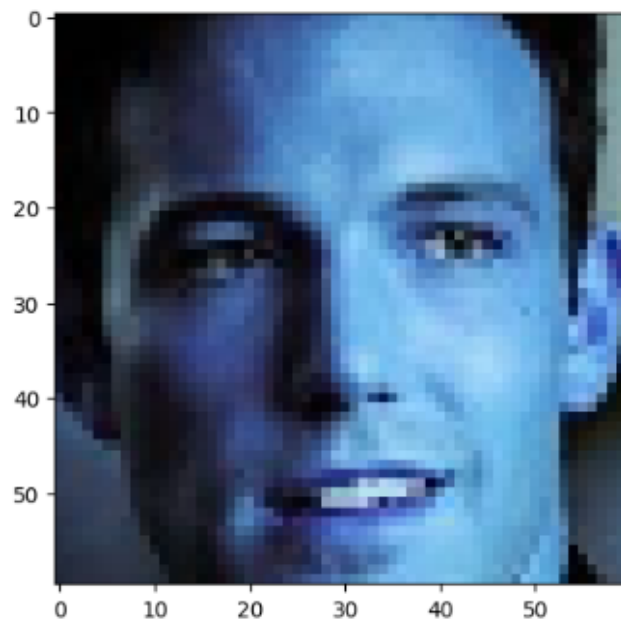
```
... 1/1 ————— 0s 400ms/step
Predicted name: ['elton_john']
Actual pred: ['elton_john']
```



```
idx = 10
img = x_test[idx]
plt.imshow(img)

enc = encode_image(anchor_model, img)
pred = neigh.predict(enc)
print("Predicted name:", le.inverse_transform(pred))
print("Actual pred: ", le.inverse_transform(y_test[idx:idx+1]))
```

1/1 ————— 0s 51ms/step
Predicted name: ['ben_afflek']
Actual pred: ['ben_afflek']



```
uploaded = files.upload()
```

```
for fn in uploaded.keys():  
    uploaded_image_path = fn  
    print(f'User uploaded file "{uploaded_image_path}"')  
    break
```

... Dosyaları Seç bencik.jpg

bencik.jpg(image/jpeg) - 3548 bytes, last modified: 25.04.2026 - 100% done

Saving bencik.jpg to bencik (4).jpg

User uploaded file "bencik (4).jpg"

```
img_uploaded = cv2.imread(uploaded_image_path)  
# Check if the image was loaded successfully  
if img_uploaded is None:  
    print(f"Error: Could not load image from {uploaded_image_path}")  
else:  
    img_rgb = cv2.cvtColor(img_uploaded, cv2.COLOR_BGR2RGB)  
    faces_uploaded = face_cascade.detectMultiScale(img_rgb, 1.3, 5)  
  
    if len(faces_uploaded) == 0:  
        print("No faces detected in the uploaded image.")  
        plt.imshow(img_rgb)  
        plt.title('No Face Detected')  
        plt.axis('off')  
        plt.show()  
    else:  
        print(f"Found {len(faces_uploaded)} face(s) in the uploaded image.")  
        x, y, w, h = faces_uploaded[0]  
  
        face_crop = img_rgb[y:y+h, x:x+w]  
        face_resized = cv2.resize(face_crop, (img_size, img_size))  
  
        face_normalized = face_resized / 255.0  
  
        plt.imshow(face_normalized)  
        plt.title('Detected Face')  
        plt.axis('off')  
        plt.show()  
  
        print(f"Normalized face shape: {face_normalized.shape}")  
  
        enc = encode_image(anchor_model, face_normalized)  
        pred = neigh.predict(enc)  
        print("Predicted name:", le.inverse_transform(pred))
```


Found 1 face(s) in the uploaded image.

Detected Face



Normalized face shape: (60, 60, 3)

1/1 ————— 0s 46ms/step

Predicted name: ['ben_afflek']



```
▶ pred_x_test = anchor_model.predict(x_test)
  pred = neigh.predict(pred_x_test)
```

```
print("Accuracy ->", np.sum(pred == y_test)/len(pred))
```

... 1/1 ————— 0s 361ms/step

Accuracy -> 0.7083333333333334

Kaynaklar ve Okumalar

Bertinetto, L., Valmadre, J., Henriques, J. F., Vedaldi, A., & Torr, P. H. (2016). Fully-convolutional siamese networks for object tracking. *European conference on computer vision (ECCV)* içinde (ss. 850-865). Springer. <https://www.robots.ox.ac.uk/~vedaldi/assets/pubs/bertinetto16fully.pdf>

Bromley, J., Guyon, I., LeCun, Y., Säckinger, E., & Shah, R. (1993). Signature verification using a "Siamese" time delay neural network. *Advances in Neural Information Processing Systems (Nips 6)*

Chollet, F. (2021). *Deep learning with Python* (2. baskı). Manning Publications.

Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press. <http://www.deeplearningbook.org> adresinden erişildi.

Hermans, A., Beyer, L., & Bastian, B. (2017). In defense of the triplet loss for person re-identification. *arXiv*. <https://arxiv.org/abs/1703.07737>

Koch, G., Zemel, R., & Salakhutdinov, R. (2015). Siamese neural networks for one-shot image recognition. *ICML Deep Learning Workshop* içinde (Cilt 2).

Schroff, F., Kalenichenko, D., & Philbin, J. (2015). FaceNet: A unified embedding for face recognition and clustering. *2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* içinde (ss. 815-823). <https://doi.org/10.1109/CVPR.2015.7298682>

**İLGİNİZE
TEŞEKKÜR
EDERİM.**



Defne BENLİAY